Ders 11 Genişletilmiş Notu: Seri İletişim, Veri Transfer Yöntemleri, Öncelikli Kesmeler, DMA ve Önbelleğe Giriş

Ders 11 Genişletilmiş Notu: Seri İletişim, Veri Transfer Yöntemleri, Öncelikli Kesmeler, DMA ve Önbelleğe Giriş

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Çerçevesi ve Pipeline Sorusu
- 2. Giriş-Çıkış Konusunun Tekrarı
- 3. Paralel ve Seri İletişim
- 4. Veri Transfer Yöntemleri
- 5. Program Kontrollü Veri Transferi
- 6. Kesme ile Başlatılan Veri Transferi
- 7. Daisy Chain Öncelik Donanımı
- 8. Paralel Öncelik Donanımı
- 9. Doğrudan Bellek Erişimi (DMA)
- 10. Giriş-Çıkış Bölümünün Örnek Problemleri
- 11. Önbelleğe Giriş: SRAM, DRAM, Hit ve Miss
- 12. Direct Mapped Önbellek
- 13. Fully Associative Önbellek
- 14. Set Associative Önbellek
- 15. Word ve Blok Kavramları

1. Dersin Çerçevesi ve Pipeline Sorusu

Hoca derse, geçen hafta başlanan *input, output and communications* konusunu hatırlatarak başladı. Ona göre bu bölüm dersin en keyifli konularındandır, çünkü bir bilgisayarın arka yüzündeki portların ve cihazlarla haberleşmesinin teknik tarafını anlatır. Kaynak olarak kullanılan Mano'nun kitabı belli bir işlemciye bağlı kalmaz. "A işlemcisi şunu, B işlemcisi bunu kullanır" demek yerine bütün alternatif çözümleri işlemciden bağımsız olarak anlatır. Bu derste de birbirinin alternatifi olan birkaç yapı göreceğiz. Bölüm sonlarındaki sorulardan seçilen örnekler de derste çözülecek.

Ders başlamadan önce gelen soru, önceki dersin pipeline kısmıyla ilgiliydi.

Soru: Pipeline'ı teorik olarak anladım, ama hangi komutun önce gireceğine ve önceki komut pipeline'dan çıkmadan onun değerinin nasıl kullanılacağına nasıl karar verildiğini oturtamadım. Kısaca üzerinden geçebilir misiniz?

Cevap: Söz konusu olan veri bağımlılığıdır (data dependency), yani bir komutun önceki komutun sonucunu beklemesi. Hoca bu bilgiyi derste yalnızca öğrencilerin dikkatini çekmek için verdiğini söyledi. Pipeline sistemi hızlandıran güzel bir teknolojidir, ama dikkat edilmesi gereken sorunları vardır. Bu sorunlar yazılım tarafında derleyici düzeyinde çözülebildiği gibi donanım tarafında da çözülebilir. Veri bağımlılığı dışında dallanma komutlarında yaşanan sorunlar da

vardır. Bütün bunların ayrıntısı Computer Architecture dersinin konusudur. Bu derste amaç, teknolojiden haberdar olmak ve sorunlu noktaları görmektir.

2. Giriş-Çıkış Konusunun Tekrarı

2.1 Çevre birimleri

Çevrimiçi çalışan giriş-çıkış cihazlarına çevre birimi (peripheral) adı verilmişti. Geçen hafta bunlara üç örnek verilmişti.

- **Klavye:** Tuşlar matris biçiminde yerleştirilir. Satırları bir decoder sürer, sütunlardan gelen bilgiyi bir multiplexer toplar. Multiplexer mikrodenetleyiciye giriş yönünde bağlıdır. Hangi tuşa basıldığı bu donanımla çözümlenir. Mikroişlemciler dersinde 4 satır ve 4 sütundan oluşan 16 tuşluk basit bir yapının deneyi yapılacak ve 8086 kodu yazılacak.
- **Sabit disk:** Bir giriş-çıkış cihazıdır.
- **Ekran (monitör):** Bir çıkış cihazıdır.

G/Ç işlemcisinden (I/O processor) daha sonra bahsedileceği de söylenmişti. Bu derste ona yalnızca bir alternatif olarak değinilecek (Bölüm 4).

2.2 G/Ç arayüzü

Bütün bu cihazları işlemciye bağlamak için bir arayüze (I/O interface) ihtiyaç vardır. Her işlemcinin farklı cihazlarla haberleşmek için farklı arayüz entegreleri vardır. Örneğin 8086 ile paralel port haberleşmesi için 8255 entegresi kullanılır. Mikroişlemciler dersinde önce bu entegrenin iç yapısı öğrenilecek, sonra donanım düzeyinde işlemciye bağlanacak.

Arayüzsüz bağlantı da mümkündür. Basit birimler, örneğin bir analog-dijital çevirici entegresi, hiçbir arayüz kullanılmadan bağlanabilir. Yine de bilgisayarın içindeki portlar genelde bir arayüz üzerinden bağlanır.

2.3 Isolated I/O ve memory-mapped I/O

Arayüzün G/Ç uzayında (I/O space) bir adresi vardır. Daha önce iki yapı görülmüştü.

- **Isolated I/O:** 8086'nın kullandığı yapıdır. Bellek için ayrı, G/Ç için ayrı bir alan vardır. Adres uçlarına ek olarak bu iki alanı seçen bir M/$\overline{IO}$ kontrol ucu bulunur ve bu uç her iki entegreye de bağlıdır. Uç 1 ise belleğe, 0 ise G/Ç uzayına erişilir. Veri yolu üzerinden aktarımın hangi kaynaktan yapılacağını bu uç seçer.
- **Memory-mapped I/O:** Aynı adres alanının bir bölgesi belleğe, bir bölgesi G/Ç'ye ayrılır. Erişim aynı komutlarla yapılır. Hem belleğe hem G/Ç'ye `MOV` komutuyla ulaşılır.

2.4 Örnek arayüz, chip select ve register select

Geçen hafta incelenen örnek arayüzün içinde dört register vardı: Port A, Port B, Control ve Status. İçeride dört register olduğu için G/Ç uzayında arka arkaya gelen dört adres bu arayüze ayrılmalıdır.

Arayüzü seçmek için chip select (CS) ucu kullanılır. Bu uç 1 olduğunda arayüz seçilir. Bu yüzden işlemciden gelen adres yolu bir adres çözümleme devresinden (address decoder) geçirilmeli ve yalnızca belli bir adres aralığında CS'yi 1 yapan işaret üretilmelidir. Arayüzün içindeki register'ları seçmek için ise iki register select ucu vardır. Bu iki ucun dört farklı kombinasyonu dört register'a erişimi sağlar.

2.5 Arayüz ile cihaz arasındaki iletişim

Buraya kadar anlatılan yapı CPU ile arayüz arasındaki bağlantıyı gösterir. Arayüz ile cihaz arasındaki iletişim için iki çözüm konuşulmuştu.

- **Strobing:** İlk ve en basit çözümdür. İletişimi başlatmak için tek tel kullanılır. Geçen hafta bu tek telin sağlıklı bir iletişim için yeterli olmadığı görülmüştü.
- **Handshaking (el sıkışma):** Daha iyi olan ikinci çözümdür. Request ve reply olmak üzere iki tel kullanılır. İki birim bu tellerle karşılıklı olarak sağlıklı bir iletişim kurabilir.

3. Paralel ve Seri İletişim

Dersin konusu cihazlar ve iletişimleri olduğuna göre önce iletişimin biçimlerine bakmak gerekir. İki biçim vardır.

Paralel iletişimde bitlerin hepsi bir birimden diğerine aynı anda aktarılır. 8 bitlik iletişim için iki birim arasında 8 tel bulunmalıdır. Genel olarak n bit, n tel üzerinden aynı anda iletilir. Yani tel sayısı aynı anda aktarılan bit sayısı kadardır. Paralel iletişim daha hızlıdır, ama daha çok tel ister. Bu yüzden genelde kısa mesafede kullanılır. Hoca, paralel portların yerini artık büyük ölçüde seri portların, özellikle hızı oldukça yüksek olan USB portlarının aldığını belirtti.

Seri iletişimde tek bir tel üzerinden bir bit aktarılır. Bitler belli bir bit hızıyla (bit rate) arka arkaya gönderilir. Tek tel yeter, ama paralel iletişimden daha yavaştır. Slayttaki örnek modemdir. Bunun dışında USB portları sayesinde seri iletişim artık pek çok alanda kullanılmaktadır.

Hoca seri haberleşme modlarının ayrıntısının Data Communications dersinin konusu olduğunu, burada kısaca konuşulduğunu söyledi. Mikroişlemciler dersinde 8251 entegresiyle hem teoride hem laboratuvarda uygulama yapılacak. 8251, *Universal Synchronous and Asynchronous Receiver and Transmitter* (USART) denilen ve her iki modda çalışabilen bir seri arayüzdür.

3.1 Asenkron seri haberleşme

Asenkron haberleşmede bitler belli bir bit hızıyla arka arkaya gönderilir. Slayttaki örnek cihaz modemdir. Bir karakterin gönderilmesi şu sırayla olur.

- Hat boşta (idle) iken bilgi aktarılmadığını gösteren durumdadır.
- Karakter gönderileceği zaman önce **start biti** gelir. Start biti her zaman lojik 0'dır ve bir karakterin geldiğini belirtir.
- Sonra **veri bitleri** gelir. En düşük anlamlı bit olan D0 önce gönderilir, ardından D1, D2, …, D7 gelir.
- İki taraf anlaşmışsa hata kontrolü için bir **parity biti** eklenir.
- Son olarak karakterin bittiğini gösteren **stop bitleri** gelir. Stop bitleri her zaman lojik 1'dir.

Kaç veri biti gönderileceği protokolde önceden belirlenir. Örnekte 8 bit verilmiştir, ama 7 bitlik ASCII kod kullanan sistemler de vardır. Stop biti sayısı da 1, 1,5 ya da 2 olabilir. Bunların ayrıntısı Mikroişlemciler dersinde görülecek.

> **Ders sonrası not:** Hoca hat boştayken gerilimin "+5 V ya da RS-232'de +12 V" olduğunu söyledi. Hattın boşta lojik 1'de olduğu ders akışından çıkar: start biti lojik 0 ile gelişini belirtir. +5 V, TTL'deki lojik 1 için doğrudur. RS-232 için ise büyük olasılıkla doğrusu şudur: Lojik 1 negatif gerilimle (tipik olarak -12 V), lojik 0 pozitif gerilimle (tipik olarak +12 V) gösterilir. Bu RS-232 bilgisi altyazıdan doğrulanmadı.

Slayttaki yapıda parity yoktur. 8 bitlik karakterin yanına bir start ve iki stop biti, yani 3 bit eklenir:

$$8 + 1 + 2 = 11 \text{ bit}$$

Görüldüğü gibi bir karakteri aktarmak için 8 bit değil 11 bit göndermek gerekir. Bit hızı biliniyorsa bir karakterin ve belli sayıda karakterin ne kadar sürede gideceği buradan hesaplanır. Bu hesabın parity'li bir örneği Bölüm 10.4'tedir.

3.2 Senkron seri haberleşme

Senkron seri haberleşmede veri çerçeveler (frame) hâlinde aktarılır. Çerçevenin başlangıcını senkronizasyon karakterleri gösterir. En başta çerçevenin başladığını belirten *start of text* gibi bir ASCII kod bulunur. Arkasından gönderilmek istenen verinin ASCII kodları art arda gelir.

3.3 UART ve USART

UART yalnızca asenkron alma ve gönderme yapar. USART'ta ise senkron ya da asenkron mod seçilebilir ve arayüz buna göre programlanır.

4. Veri Transfer Yöntemleri

Çevre birimlerine ve çevre birimlerinden veri aktarımı (*modes of transfer*) için üç temel yöntem vardır.

- **Program kontrolünde transfer (data transfer under program control):** En ilkel yöntemdir. Bilgisayar mimarisinde kullanılmaz, ama tek işi olan özel (dedicated) bir donanım tasarlandığında kullanılabilir. Hocanın örneği, ortamın aydınlığını izleyip hava kararduğında ışığı açan, aydınlandığında söndüren bir devredir. Bu yöntemde işlemci yalnızca bu iş için bloke edilir. Bu yüzden bilgisayarın içinde kullanılamaz.
- **Kesme ile başlatılan transfer (interrupt-initiated data transfer):** Bilgisayarın içinde tercih edilen ve kullanılan çözümdür.
- **Doğrudan bellek erişimi (direct memory access, DMA):** Büyük veri bloklarının aktarılması gerektiğinde aktarımı hızlandırmak için kullanılır.

Hoca buna dördüncü bir çözüm olarak G/Ç işlemcisini de ekledi. Bu çözüm özellikle mainframe'lerde, büyük sistemlerde kullanılır, PC için gerekli değildir.

5. Program Kontrollü Veri Transferi

> **Hocanın vurgusu:** Bu yapıyı dikkatli dinleyin. Mikroişlemciler dersinde sıkça kullanılacak, nasıl çalıştığını anlarsanız sonraki konuları daha hızlı kavrarsınız.

5.1 Yapı

Sistem üç modülden oluşur: işlemci, arayüz ve G/Ç cihazı. G/Ç yolunun (I/O bus) çiziliş yönü bu cihazın bir giriş cihazı olduğunu gösterir. Cihazdan okunan veri önce arayüzün içindeki data register'a yazılır, sonra veri yolu üzerinden işlemciye aktarılır. Bu yüzden arayüzden işlemciye giden veri yolu yalnızca işlemci yönünde çizilmiştir.

CPU ile arayüz arasında adres yolu ve I/O read, I/O write gibi kontrol işaretleri vardır. Şekilde adres yolu arayüze doğrudan gidiyor gibi görünür, ama araya bir adres çözümleme devresi girmeli ve bu devre arayüzü seçecek chip select işaretini üretmelidir.

Arayüzün içinde basit olsun diye yalnızca iki register vardır: data register ve status register. İki register olduğu için arayüz, işlemcinin G/Ç uzayında arka arkaya iki adres kaplar. Hoca bu adresleri örnek olarak şöyle seçti:

- Data register: A8h
- Status register: A9h

5.2 Veri arayüze nasıl gelir?

İşlem adımları sırayla şöyledir.

1. G/Ç cihazı aktarmak istediği veriyi G/Ç yoluna bırakır.
2. Ardından ready işaretini aktif yapar.
3. G/Ç yolundaki veri arayüzün data register'ına yazılır.
4. Veri kaydedildikten sonra arayüz acknowledge işaretini aktif yapar.

Veri artık arayüzdedir, ama amaç onu CPU'ya aktarmaktır. Arayüzde hazır veri olduğunu status register'daki bir bayrak biti (flag) gösterir. Örnekte bu bit en düşük anlamlı bittir. Arayüzde hazır veri varsa bayrak 1 yapılır, yani "data register dolu" denir.

5.3 CPU tarafı: yoklama döngüsü

CPU, status register'ı sürekli okuyarak hazır veri olup olmadığını denetler. Akış diyagramı şöyledir: status register oku, bayrağı denetle, bayrak 0 ise başa dön, 1 ise data register'ı oku,

veriyi kullan, transfer bitmediyse yine başa dön. Hocanın söylediği komutlarla bu şöyle yazılır. Atlama komutu derste adıyla söylenmedi, yalnızca “sıfırsa başa dönüyor” denildi (ara adım eklendi).

```
KONTROL: IN   AL, 0A9h    ; status register'ı oku
         TEST AL, 01h     ; en düşük anlamlı biti denetle
         JZ   KONTROL     ; 0 ise hazır veri yok, başa dön
         IN   AL, 0A8h    ; data register'ı oku
         ; veriyi belleğe yaz ya da üzerinde işlem yap
         ; transfer bitmediyse KONTROL'e dön
```

Okunan veri belleğe kaydedilebilir ya da üzerinde başka bir işlem yapılabilir. Transfer tamamlanmadıysa çevrim yeniden başlar.

Yöntemin sorunu, bir veri bloğunu aktarmak için işlemcinin sürekli bu döngüde kalması, yani bu işe bloke olmasıdır. Hocanın örneği klavyedir. İnsanın yazma hızıyla bilgisayarın çalışma hızını karşılaştırın. Siz iki tuşa basana kadar arka taraftaki yazılım bunu belki binlerce kez kontrol eder. Bu yüzden yöntem verimli değildir. Yalnızca özel amaçlı, çok küçük bir devrede kullanılması düşünülebilir.

6. Kesme ile Başlatılan Veri Transferi

İkinci çözümün fikri şudur: İşlemciyi sürekli bir döngüde kilitlemek yerine kendi işlerini yapmasına izin verilir ve yalnızca veri geldiğinde bu durum işlemciye kesme (interrupt) ile bildirilir.

Önceki şekilde gösterilmeyen, ama bu çözümde bulunan bir bağlantı vardır. Arayüzün data register’ına veri yerleştirilip bayrak biti 1 olduğunda işlemciye bir interrupt request işareti gider. Bu işaret yalnızca veri hazır olduğunda gider. İşlemci kesmeyi alınca yürütmekte olduğu komutu tamamlar ve ilgili alt programa (subroutine) dallanır. Arayüzü okumak için gereken satırları, yani “data register’dan oku, okuduğunu şuraya yaz” gibi işlemleri bu alt programa biz yazarız.

Birden fazla cihaz kesme isteyebileceği için kesmeli transferde bir önceliklendirme lojiğine ihtiyaç vardır. Bunun için iki alternatif anlatıldı: seri çalışan daisy chain önceliği ve paralel çalışan öncelik donanımı.

7. Daisy Chain Öncelik Donanımı

7.1 Yapı

Örnekte üç cihaz vardır: Device 0, Device 1 ve Device 2. Bu cihazların kesme istekleri (interrupt request) üç girişli bir OR kapısıyla toplanır. İsteklerin aktif seviyesi lojik 1’dir. Birimlerden herhangi birinden istek gelirse CPU’ya interrupt request gider.

İşlemci isteği alınca önce yürüttüğü komutu tamamlar ve “kesme işlemi için hazırım” anlamında bir interrupt acknowledge işareti gönderir. Bu işaret de lojik 1 kabul edilmiştir. Her bloğun bir priority in (PI) girişi ve priority out (PO) çıkışı vardır. Acknowledge işareti CPU’dan ilk bloğun PI girişine gelir, sonra bloklar arasında PO’dan PI’ya sırayla aktarılır.

> **Hocanın vurgusu:** Daisy chain’de acknowledge işaretini ilk alan birim en yüksek önceliğe sahiptir. Burada bu birim en soldaki Device 0’dır. En sağdaki birim en düşük önceliğe sahiptir.

Bir birim veri yoluna vektör adresini (VAD) bırakır. Vektör adresi, kesme alt programının numarasını gösteren adrestir ve ilgili alt programa dallanılmasını sağlar.

7.2 Örnekler

Aşağıda istekler sırasıyla Device 0, Device 1, Device 2 için yazılmıştır.

İstekler 1, 0, 0: Yalnızca en yüksek öncelikli birim istekte bulunmuştur. Acknowledge’ı ilk alan birim Device 0’dır, kendisinde de istek vardır. Hizmet Device 0’a verilir ve onun vektör adresine dallanılır.

İstekler 1, 1, 0: Device 0 ve Device 1 istekte bulunmuştur. Acknowledge'ı ilk alan Device 0 olduğu için vektör adresini yine o bırakır. Device 0'ın PO çıkışı 0 olur, yani acknowledge yanındaki birime aktarılmaz. Device 1'in çıkışı da 0 olur. Yalnızca Device 0'a hizmet verilir.

İstekler 0, 1, 1: Device 0'da istek yoktur. CPU'nun gönderdiği lojik 1'i PI girişinde görür, ama istek kendisinden gelmediği için bu işareti PO çıkışından sağındaki birime aktarır. Acknowledge, Device 1'e gider. Device 1 işlemcinin hazır olduğunu görür. Kendisinde de istek olduğu için vektör adresini işlemcinin veri yoluna bırakır ve acknowledge'ı Device 2'ye aktarmaz. Device 2'de istek olsa da işlemcinin o anki müsaitliğinden Device 1 faydalanır. Device 2'ye sıra, Device 1 veri gönderme işini tamamlayıp isteği 0'a döndükten sonra gelir.

7.3 Bloğun iç yapısı ve tablo

Bir bloğun dış uçları PI, PO ve interrupt request çıkışıdır. İçinde bir SR latch, bir gecikme (delay) lojiği, iki AND kapısı ve vektör adresinin bırakıldığı, enable ile denetlenen bir register vardır. RF, kesme isteğini tutan noktadır. RF = 1 ise bu birimin kesme isteği vardır. CPU'ya giden interrupt request çıkışı da bu noktadan alınır.

Çalışmayı özetleyen tablo şöyledir. PI ve RF giriş, PO ve Enable çıkıştır. İlk satırı hoca okudu, diğer satırları derste bir öğrenciyle birlikte yorumladı.

PI	RF	PO	Enable	Anlamı
0	0	0	0	Acknowledge gelmedi, bu birimin isteği de yok
0	1	0	0	Bu birimin isteği var, ama acknowledge ona ulaşmadı
1	0	1	0	Acknowledge geldi, bu birimin isteği yok, sağa aktarılır
1	1	0	1	Acknowledge geldi, bu birimin isteği var, vektör adresi bırakılır

Birinci satır: PI = 0 olması işlemciden acknowledge gelmediğini, yani kesme için hazır işareti görünmediğini gösterir. RF = 0 ise bu birimin isteği yoktur. Birim acknowledge almadığı için sağındaki birime de 0 aktarır (PO = 0). Enable = 0'dır, çünkü bu birimin vektör adresi, yani kesme alt programının adresi yola bırakılmamalıdır.

İkinci satır: RF = 1'dir, yani birimin isteği vardır, ama PI = 0 olduğu için işlemci bu isteği değerlendirmez. Hoca bunun nedenini sordu.

Soru: (Hoca sordu.) Birim istek göndermişken işlemcinin ona cevap vermemesi nasıl yorumlanır?

Cevap: (Bir öğrenci yanıtladı.) Bu birimden daha öncelikli bir kesme isteği vardır, işlemci onu değerlendirmektedir.

Hoca cevabı onayladı ve bu durumun örneğin Device 2'nin başına gelebileceğini söyledi. Device 2 istek gönderir, ama kendinden önceki birimlerde de istek varsa öncelik onlarda olduğu için CPU onlara hizmet verir ve Device 2'nin PI girişi 0 kalır.

Üçüncü satır: Sıra bu birimdedir (PI = 1), ama isteği yoktur (RF = 0). Bu yüzden acknowledge'ı sağındaki birime aktarır (PO = 1). Hoca bunun nasıl olabileceğini istekleri 0, 0, 1 alarak gösterdi. CPU acknowledge göndermiştir, ama istek bu birimden gelmemiştir. Demek ki istek daha düşük öncelikli bir birimdendir ve işaret ona aktarılır.

Dördüncü satır: PI = 1 ve RF = 1'dir. Birimin isteği vardır ve sıra ona gelmiştir. Sağındaki birime acknowledge gönderilmez (PO = 0), çünkü veriyi aktarmak isteyen birim budur. Enable = 1 olması kesme isteğinin gerçekleşmesi, yani kesme alt programı adresinin gönderilmesi demektir.

> **Ders sonrası not:** Hoca üçüncü satır için önce "1, 0, 1 olsun ya da 0, 0, 1 olsun" dedi. Python ile yapılan zincir benzetimi yalnızca 0, 0, 1 durumunun bu satırı ikinci birimde oluşturduğunu gösterdi. 1, 0, 1 durumunda Device 0 hizmet alır, Device 1'in PI girişi 0 olur ve Device 1 birinci satıra düşer. Yukarıda geçen 1, 0, 0; 1, 1, 0; 0, 1, 1 örneklerinin sonuçları ve tablonun dört satırı da aynı benzetimle doğrulandı.

Tablodan iki çıkışın Boole ifadesi okunabilir (ara adım eklendi). Bloktaki iki AND kapısı da bu iki çıkışla uyumludur.

$$PO = PI \cdot \overline{RF}, \qquad Enable = PI \cdot RF$$

Daisy chain seri çalışan bir yapıdır. Bu yüzden biraz daha yavaştır, ama bir öncelik lojiği oluşturur. Bazı işlemciler bu çözümü, bazıları diğerini kullanır.

8. Paralel Öncelik Donanımı

Hocaya göre ikinci çözüm biraz daha kolaydır. Örnek modelde dört kaynaktan kesme isteği gelebilir. İstekler bir **interrupt register**'a yerleştirilir. En yüksek öncelikli kaynak 3 numaralı girişe, en düşük öncelikli kaynak register'ın en düşük anlamlı bitine bağlıdır.

Yanında bir de **mask register** vardır. Bu register'la istenen kaynaklar yazılımla kapatılabilir. Mask register'ın ilgili biti 0 olduğunda o birimden gelen istek AND kapılarıyla engellenir. Bit 1 olduğunda istek geçer.

Dört kaynaktan gelen istekler 4'ten 2'ye bir öncelik kodlayıcıya (priority encoder) girer. Öncelik kodlayıcı dersin ilk haftasında kombinasyonel devreler anlatılırken görülmüştü. Girişleri D0-D3'tür, D3 en yüksek önceliklidir. Çıkışları iki bitlik ikili kod A1 A0 ile bir geçerlilik biti (valid bit) V'dir.

D3	D2	D1	D0	A1	A0	V	Anlamı
0	0	0	0	X	X	0	İstek yok, A1 A0 önemsiz
0	0	0	1	0	0	1	Kaynak 0
0	0	1	X	0	1	1	Kaynak 1, D0'ın değeri önemsiz
0	1	X	X	1	0	1	Kaynak 2, daha düşük öncelikliler önemsiz

D3	D2	D1	D0	A1	A0	V	Anlamı
1	X	X	X	1	1	1	Kaynak 3, daha düşük öncelikliler önemsiz

Tablo python ile girişlerin 16 kombinasyonunda doğrulandı. Hepsi 0 iken V = 0'dır ve A1 A0'ın ne gösterdiği önemli değildir. D1'den istek gelmişse D1 daha öncelikli olduğu için D0'ın ne dediğinin önemi yoktur. Aynı mantık D2 ve D3 satırlarında da geçerlidir.

Kodlayıcının iki bitlik çıkışı **vektör adresinin** iki bitini oluşturur. A1 yüksek, A0 düşük anlamlı olacak şekilde bağlanır. Vektör adresinin yüksek anlamlı 6 biti 0'dır. Böylece her cihazın vektör adresi yalnızca alttaki iki bitle değişir.

Cihaz	A1 A0	Vektör adresi
Device 0	00	00h
Device 1	01	01h
Device 2	10	02h (ara adım eklendi)
Device 3	11	03h

V biti 1 olduğu için aynı zamanda CPU'ya kesme isteği olarak gönderilir (*interrupt to CPU*).

> **Ders sonrası not:** Hoca Device 3 için vektör adresini "0123", ardından "013 hex" diye okudu. Yüksek 6 bit 0 ve alt iki bit 11 olduğuna göre doğrusu `0000 0011`, yani 03h'tir. Python ile doğrulandı.

Soru: Assembly'den maskelenemeyen kesmeleri hatırlıyorum. Maskeleme donanımla mı yapılıyor, yazılımla mı?

Cevap: Maskelenemeyen kesmeler kritik kesmelerdir ve hiçbir şekilde maskelenemez. Buradaki yapıda ise bunun için elbette bir donanım vardır, ama yapı programlanabilir. Mask register istenildiği gibi yazılımla programlanır.

9. Doğrudan Bellek Erişimi (DMA)

9.1 Bir anı

Hoca konuya bir anıyla başladı. Yıllar önce hazırlık sınıfındaki iki öğrenci gelip işlemciyi güçlendirecek bir çözüm düşündüklerini söylemiş. Fikirleri şuymuş: Diskten belleğe aktarımı hızlandırmak için bir birim tasarlamak, işlemciyi dışarıda bırakmak ve diskten okunan veriyi doğrudan RAM'e aktarmak. Henüz bilgisayar donanımı okumamış olmalarına rağmen böyle düşünmeleri hocanın hoşuna gitmiş. Ama bu yapı zaten vardı. Hoca onlara DMA ile ilgili bir doküman vermiş.

9.2 Amaç ve yolun devralınması

DMA, ana bellek (RAM) ile G/Ç sistemi arasındaki veri transferini hızlandırarak sistem başarımını artırır. Bunu CPU'yu devre dışı bırakıp doğrudan aktarım yaparak sağlar. Elimizdeki sistemlerin tamamı CPU tarafından yönetildiği için CPU'yu devre dışı bırakacak iki uç gerekir: **bus request (BR)** ve **bus grant (BG)**.

- DMA denetleyicisinden gelen BR işaretini alan CPU, bir doğrudan bellek erişimi talebi olduğunu anlar ve yolun kontrolünü bırakır.
- CPU'nun kontrol ettiği adres yolu, veri yolu, read ve write işaretleri yüksek empedans (high-Z) durumuna, yani üçüncü konuma çekilir.

- CPU, BG'yi 1 yapar. Bu, yolun kontrolünü bıraktığı ve bundan sonra yolu DMA denetleyicisinin yöneteceği anlamına gelir.

  Hocanın vurgusu: DMA'da CPU devre dışı bırakılır. Bu yüzden bus request ve bus grant işaretlerinin önemli bir rolü vardır.

9.3 DMA denetleyicisi

DMA denetleyicisi de önceki örneklerdeki gibi bir arayüzdür. Sol taraftaki uçlar CPU ile haberleşme, sağ taraftaki uçlar G/Ç cihazıyla haberleşme içindir.

- **CPU tarafı:** DMA select (DS, diğer adıyla chip select), register select (RS), read, write, bus request, bus grant ve interrupt.
- **G/Ç tarafı:** DMA request ve DMA acknowledge.

İçeride üç register vardır. CPU devre dışı kaldığına göre adres bilgisini, aktarılacak veri miktarını ve aktarımın denetimini artık denetleyici sağlamalıdır.

- **Adres register'ı:** Verinin bellekte hangi adrese yazılacağını ya da bellekteki hangi verinin dışarı aktarılacağını tutar. Her transferden sonra 1 artırılır ve böylece sonraki bellek adresini gösterir.
- **Word count register'ı:** Aktarılacak verinin toplam miktarını tutar. Her transferden, yani her byte aktarıldıktan sonra 1 azaltılır.
- **Control register:** Veri transferini denetler.

Örnekte tek bir register select ucu vardır. Hocaya göre register sayısı üç gibi görünse de bazı register'lara sıralı erişildiği için sanki iki register adresleniyor gibidir. Örneğin aynı adres ilk okunduğunda word count register, ikinci okunduğunda control register okunuyor olabilir. Bazı arayüzlerde böyle sıralı okuma vardır.

9.4 Bütün sistem ve işlem adımları

Büyük resimde işlemci, DMA denetleyicisi, ana bellek ve büyük miktarda veri okunan bir G/Ç cihazı vardır. Bu cihaz örneğin iki yönlü aktarım yapılan bir disk olabilir. Bu yüzden veri yolu çift yönlü çizilmiştir. Şekle göre cihaz RAM ile veri yolu üzerinden doğrudan iletişim kurar. Bunu sağlayan birim DMA denetleyicisidir.

Hocanın anlattığı sıra şöyledir.

1. G/Ç cihazı veri göndermeye hazır olduğunu DMA denetleyicisine **DMA request** ile bildirir.
2. Denetleyicinin adres ve word count register'ları ayarlanır. Bu bilgileri CPU yazar. İç register'lara, daha önce A8h ve A9h örneğindeki gibi G/Ç adresleri üzerinden erişilir.
3. Her şey hazır olunca denetleyici CPU'ya **bus request** gönderir.
4. CPU adres ve veri yolunu yüksek empedansa çekip yolun kontrolünü bırakır ve **bus grant**'i 1 yapar. Denetleyici artık sistemin yöneticisidir.
5. Denetleyici adresi adres yoluna bırakır, yazma mı okuma mı olduğunu seçer ve G/Ç cihazına **DMA acknowledge** gönderir.
6. Acknowledge aktif olduğu sürece cihaz transferi yapar. Her transferden sonra adres register'ı 1 artar, word count register'ı 1 azalır ve bir sonraki byte aktarılır.

Hoca register yüklemeyi şu tür komutlarla örnekledi:

```
OUT 0A8h, AL    ; AL'deki değeri adres register'ına yaz
OUT 0A9h, AL    ; AL yeniden ayarlandıktan sonra word count register'ına yaz
```

Ders sonrası not: Hoca word count için "AL'yi ya da AH register'ını ayarlayıp" A9h adresine yüksek anlamlı byte'taki değeri atmaktan söz etti. 8 bitlik `OUT` komutunun kaynağı AL olmalıdır. `OUT 0A9h, AH` satırı assembler'dan (GNU as, 16 bit kipi) "operand type mismatch" hatası aldı, `OUT 0A8h, AL` ise derlendi. AH'deki değer gönderilecekse önce AL'ye taşınmalıdır (ara adım eklendi). Kısa nottaki `OUT 0A9h, AH` yazımı da bu yüzden geçersizdir.

> **Ders sonrası not:** Hoca önce DMA request'i, sonra CPU'nun register yüklemesini anlattı. Altyazıdan doğrulanmayan bir bilgi olarak, büyük olasılıkla kitaptaki olağan sıra terstir: CPU denetleyiciyi önce ilklendirir, cihaz DMA request'i sonra gönderir.

Aktarım cihazdan belleğe, veri yolu üzerinden doğrudan yapılır. Her byte önce bir CPU register'ına alınıp sonra belleğe yazılmaz, cihazdan gelen veri doğrudan belleğe yazılır. DMA'nın çok daha hızlı olmasının nedeni budur.

10. Giriş-Çıkış Bölümünün Örnek Problemleri

10.1 Problem 1: Arayüzü 240-243 adreslerine yerleştirmek

Verilen: İçinde dört register bulunan örnek arayüz, G/Ç uzayında onluk tabanda 240, 241, 242 ve 243 adreslerine yerleştirilecektir. G/Ç adresi 8 bittir. CPU ile arayüz arasındaki dış devre istenmektedir.

Adresleri ikiliye çevirelim:

Onluk	A7 A6 A5 A4	A3 A2	A1 A0
240	1 1 1 1	0 0	0 0
241	1 1 1 1	0 0	0 1
242	1 1 1 1	0 0	1 0
243	1 1 1 1	0 0	1 1

Yalnızca en düşük anlamlı iki bit değişir. Bu bitler register select için ayrılır. Değişmeyen A7-A2 uçları adres çözümleme devresine girer.

Devre:

- A7, A6, A5, A4 uçlarının hepsi 1 olmalıdır. Bu uçlar bir AND kapısına bağlanır. AND'in çıkışı yalnızca bütün girişleri 1 iken 1 olur.
- A3 ve A2 uçları 0 olmalıdır. Bu uçlar bir NOR kapısına bağlanır. Her iki giriş 0 iken çıkış 1 olur.
- İki kapının çıkışı bir AND kapısına girer ve çıkışı CS'dir.
- A1 ve A0 uçları arayüzün RS1 ve RS0 girişlerine sırayla bağlanır.
- CPU'nun veri yolu arayüze doğrudan bağlanır.

Python ile 8 bitlik 256 adresin hepsi denendi. CS yalnızca 240, 241, 242 ve 243 için 1 oluyor. Hocanın garanti edilmesini istediği koşul budur.

10.2 Problem 2a: Adres çözücü olmadan kaç arayüz bağlanır?

Verilen: Adres 16 bittir. Her arayüzün chip select ucu farklı bir adres ucuna bağlanacaktır, yani adres çözümleme devresinin maliyetinden kurtulmak istenmektedir.

Çözüm: Adres yolunun en düşük iki biti (A1, A0) iç register'ları seçmek için ayrılır ve bütün arayüzlerin ortak register select uçlarına bağlanır. Kalan her adres ucu bir arayüzün chip select girişine bağlanır. Örneğin birinci arayüze A15, ikinci arayüze A14 gider.

$$16 - 2 = 14 \text{ uç} \Rightarrow \text{en fazla } 14 \text{ arayüz}$$

Fakat aynı anda yalnızca bir arayüz aktif olmalıdır. Aksi hâlde veri yoluna aynı anda veri bırakmak isteyen birden fazla birim olur. Bu kısıt nedeniyle belli adres bölgeleri kullanılamaz hâle gelir. A15'e bağlı arayüz seçilirken A14'ten A2'ye kadar bütün uçlar 0 olmalıdır. Adresin üst bitleri `1000 0000 0000 00` olur, son iki bit register seçer. Hexadecimal olarak ilk iki arayüzün adresleri şöyledir:

Arayüz	CS ucu	Adres aralığı
Interface 1	A15	8000h, 8001h, 8002h, 8003h
Interface 2	A14	4000h, 4001h, 4002h, 4003h

Arayüz	CS ucu	Adres aralığı

Port A register'ı 8000h'tedir, diğer üç register onu izler.

Soru: Neden 8000 oldu?

Cevap: Birinci arayüzün CS ucu A15'e doğrudan bağlı olduğu için bu arayüz A15 = 1 iken seçilir. Diğerlerinin seçilmemesi için kalan üst uçlar 0 olmalıdır. Bu 16 bitlik adres hexadecimal olarak 8000h eder.

Soru: (Hoca sordu.) A14'e bağlı Interface 2'nin adres aralığı nedir?

Cevap: (Bir öğrenci önce adresi ikiye bölmeyi önerdi.) A14 = 1, diğerleri 0 olacağı için aralık 4000h ile başlar ve 4003h'te biter. Hoca "ikiye bölme" diye bir kural konmamasını istedi. Öğrenci düşüncesini 15'ten 14'e bir basamak kaydırma olarak açıkladığında bunun böyle düşünülebileceğini söyledi.

> **Ders sonrası not:** Hoca adresleri okurken "80000, 800001, 80001" ve "40001, 40002, 40003" dedi. 16 bitlik adres 4 hex hanedir. Doğrusu 8000h-8003h ve 4000h-4003h'tir. Python ile doğrulandı. Ayrıca hoca 8002h ve 8003h için bir yerde "port C register" dedi. Dersteki örnek arayüzün dört register'ı Port A, Port B, Control ve Status olduğu için Port C bu arayüzde yoktur.

10.3 Problem 2b: Adres çözücü ile kaç arayüz bağlanır?

Bu kez A15-A2 uçları bir adres çözümleme devresinden geçirilir (*fully decoded*). Yüksek anlamlı 14 bit kullanıldığı için aynı tipten

$$2^{14} = 16384$$

arayüz bağlanabilir. Hoca sonucu 2^{14} olarak bıraktı, sayısal değer python ile hesaplandı (ara adım eklendi).

Üçüncü soru öğrencilere bırakıldı.

10.4 Problem 4: Bir karakterin seri iletim süresi

Hoca soruyu önce öğrencilere bırakacaktı, sonra tahtada çözdü. Slayttaki "baud rate" burada bit hızı olarak düşünülmelidir.

Verilen: Bit hızı 57600 bit/s'dir. ASCII 'A' karakteri asenkron olarak, even parity ve iki stop bitiyle gönderilecektir. 'A' karakterinin ASCII kodu 41h'tir.

Adım 1, ikiliye çevirme: 41h = `0100 0001`.

Adım 2, gönderim sırası: En düşük anlamlı bitten başlanır.

Bit	Start	D0	D1	D2	D3	D4	D5	D6	D7	Parity	Stop	Stop
Değer	0	1	0	0	0	0	0	1	0	0	1	1

Adım 3, parity: Parity biti hata denetimi içindir, karakterin karşıya doğru aktarılıp aktarılmadığını denetlemeyi hedefler. Even parity, parity biti dahil birlerin sayısının çift olmasını garanti eder. Veride iki tane 1 olduğu için parity biti 0'dır.

Adım 4, toplam bit: 8 veri bitine 1 start, 1 parity ve 2 stop biti, yani 4 bit eklenir.

$$8 + 4 = 12 \text{ bit}$$

Adım 5, süre: Bir bitin gönderilme süresi $1/57600$ saniyedir. Hoca sonucu bu biçimde bıraktı. Sayısal değerler python ile hesaplandı (ara adım eklendi).

$$t_{\text{bit}} = \frac{1}{57600}\ \text{s} \approx 17{,}36\ \mu s$$

$$t = 12 \times \frac{1}{57600}\ \text{s} \approx 208{,}33\ \mu s$$

Ders sonrası not: Hoca çözümde “stop bitiyle başladınız” ve “bir tane stop biti ekledik” dedi. Bölüm 3.1’de de anlatıldığı gibi karakter **start** bitiyle başlar ve eklenen bitler 1 start + 1 parity + 2 stop bitidir. 41h’yi ayrıca 9 haneli olarak `010000001` diye okudu. Doğrusu 8 bitlik `01000001`’dir. Hocanın saydığı bit dizisi (D0 = 1, D1-D5 = 0, D6 = 1, D7 = 0) doğrudur. Python ile doğrulandı.

Ders sonrası not: Hoca parity için “bu 8 bitlik sayının içindeki birlerin sayısı” dedi. Parity biti dahil sayıldığında burada 9 bit vardır. Ayrıca hoca parity’nin yalnız tek bit hatasını bildirebildiğini, birden fazla bitte hata olursa bildiremediğini söyledi. Python ile bu çerçevede denendiğinde 1 ve 3 bitlik hataların hepsi, 2 bitlik hataların ise hiçbiri yakalanmadı. Yani parity tek sayıda bit hatasını yakalar, çift sayıda bit hatasını yakalayamaz. Hocanın “ilkel bir hata denetimi” değerlendirmesi bununla uyumludur.

11. Önbelleğe Giriş: SRAM, DRAM, Hit ve Miss

Yeni konu önbellek (cache memory) konusudur. Hoca, bilgisayar satın alırken RAM’ine ve önbelleğine bakılmasına rağmen bu teknolojilerin mantığının da bilinmesi gerektiğini söyledi. Bu derste yaklaşık bir saatlik bir giriş yapıldı, konu sonraki derste sürecek. Kaynak yine Mano’nun kitabının 14. bölümüdür.

11.1 SRAM ve DRAM

Önbellek bloğu SRAM (statik RAM) teknolojisiyle, ana bellek ise DRAM (dinamik RAM) ile yapılır.

	SRAM	DRAM
Kullanım	Önbellek	Ana bellek
İçindeki eleman	SR latch	Kapasitör
Hız	Daha hızlı	Hocaya göre neredeyse onda biri hızında
Maliyet	Çok daha pahalı	Daha ucuz

Hoca bir RAM modülünün zaten pahalı olduğunu, statik RAM’in bunun çok üstünde fiyatlara mal olduğunu söyledi.

Soru: Kapasitör neden daha yavaş?

Cevap: DRAM anlatılırken refresh işlemine ihtiyaç duyduğundan bahsedilmişti. Yavaşlıktan kasıt, bir adres verildiğinde oradaki verinin hazırlanıp getirilmesinin daha uzun sürmesidir. Bunun nedeni bu yenilemeler ve kullanılan teknolojidir.

Soru: Yol uzun olduğu için mi?

Cevap: Hayır, yolla ilgisi yoktur. Neden tamamen içerideki kapasitörlerle kurulmuş yapıdır. SRAM ise SR latch kullandığı için yapı olarak çok daha hızlı cevap üretir.

Ders sonrası not: Hoca yavaşlığı refresh’e ve kapasitörlü yapıya bağladı. Altyazıdan doğrulanmayan bir bilgi olarak, büyük olasılıkla erişim süresindeki asıl pay kapasitördeki küçük yükün okunması gibi işlemlerdendir, refresh ise yalnızca belli aralıklarla araya girer.

Hocanın vurgusu: Maliyet yüzünden önbellek ana belleğin yalnızca küçük bir parçası kadar, örneğin onda biri, belki yirmide biri büyüklüğünde tutulur.

> **Ders sonrası not:** Altyazıdan doğrulanmayan bir bilgi olarak, büyük olasılıkla günümüz bilgisayarlarında oran bundan çok daha küçüktür (birkaç MB önbellek, birkaç GB ana bellek).

Soru: Ana belleğin de tamamen SRAM olduğu kullanımlar var mı?

Cevap: Hoca hiç duymadığını söyledi. Mümkündür, ama maliyet çok büyük olacağı için tercih edilmez.

11.2 Hit ve miss

CPU aradığı veriyi önce önbellekte arar. Aranan verinin önbellekte bulunmasına **hit**, bulunamamasına **miss** denir. Miss olursa aynı adres bu kez ana bellekte aranır ve veri oradan okunur. İlk aramada veri önbellekte bulunursa veriye hocanın verdiği orana göre 10 kat daha hızlı ulaşılır. Önbellek yapısı bellekten okumayı hızlandırmak için geliştirilmiştir.

Önbellek, komut önbelleği (instruction cache) ve veri önbelleği (data cache) olarak iki ayrı yapı şeklinde de kullanılabilir.

11.3 Üç tasarım seçeneği

Önbellek tasarımında üç eşleme tekniği vardır: direct mapped, fully associative ve set associative.

> **Hocanın vurgusu:** Üç yapıyla da değişik örnekler yapılacak. Hepsi matematiktir. Matematiğini anlarsanız çok rahat çözersiniz.

Soru: Önbelleğin sanal bir versiyonu da oluyor mu? Thread denilen şey önbellek mi?

Cevap: Thread bambaşka bir konudur. Ama önbellek fikrini kullanan farklı yazılım yapıları vardır. Örneğin veritabanı tasarımında da bu eşleme tekniklerinin kullanıldığını gördüğünü söyledi. Önce yapının öğrenilmesini, karşılaştırmaların ilerideki derslerde yapılabileceğini belirtti.

12. Direct Mapped Önbellek

Verilen: Adres yolu 10 bittir. Ana bellekte her satır 4 byte, yani $4 \times 8 = 32$ bit tutar. Bellek byte adresli olduğu için ilk satırın adresi 0, ikinci satırın adresi 4'tür. Önbellek 8 satırdır. Hoca bu değerlerin örnek olsun diye çok küçük tutulduğunu söyledi.

Adres, sağdan sola byte offset, index ve tag alanlarına bölünür.

- **Byte offset:** Satırdaki 4 byte'ı (byte 0, 1, 2, 3) seçer. $\log_2 4 = 2$ bit.
- **Index:** Önbelleğin 8 satırını seçer. $\log_2 8 = 3$ bit. Satırlar 0'dan 7'ye kadar numaralanır.
- **Tag:** Kalan bitlerdir. $10 - 3 - 2 = 5$ bit.

Tag (5 bit)	Index (3 bit)	Byte offset (2 bit)

Örnek: Ana bellekteki 4 adresindeki sayı önbelleğe taşınacak olsun.

$$4 = \underbrace{00000}_{\text{tag}}\ \underbrace{001}_{\text{index}}\ \underbrace{00}_{\text{offset}}$$

Index alanı 001 olduğu için veri ancak önbelleğin 001 satırına konabilir. Yüksek anlamlı 5 bit, yani 00000, tag olarak veriyle birlikte aynı satıra yazılır.

Tag'e neden ihtiyaç var? Ana bellekte index'i 001 olan başka adresler de vardır. Hepsi önbellekte aynı satıra karşılık gelir. O satırda hangisinin durduğu ancak tag alanına bakılarak söylenebilir. Bu yüzden önbellek yalnızca veriyi değil, verinin adres bilgisini de tutar. Tag 5 bit olduğu için bu örnekte aynı index'e düşen 32 satır vardır (ara adım eklendi, $2^5 = 32$).

Dezavantaj: Aynı index'e sahip adreslerden önbellekte aynı anda yalnızca biri tutulabilir.

Hocanın vurgusu: Direct mapped önbelleğin çalışma mantığı budur, çok önemlidir.

Soru: Index'i 001 olan iki veriyi de kullanmamız gerektiğinde birini kullanamıyor muyuz?

Cevap: Kullanamazsınız. Biri diğerinin üzerine yazar (overwrite). Önbellekte yalnızca en son kullanılan güncel olan tutulur.

13. Fully Associative Önbellek

Bu teknikte index kullanılmaz. 10 bitlik adres yalnızca iki alana ayrılır.

Tag (8 bit)	Byte offset (2 bit)

Index olmadığı için veri önbelleğe sırayla taşınır. Örnekte önbellek başta temizlenmiştir, bütün satırlar boştur. Taşınacak ilk adres ilk satıra yazılır ve 8 bitlik tag'i de veriyle birlikte taşınır. Sonraki adres için aynı işlem yapılır.

Önbellek dolunca hangi satırın üzerine yazılacağına bir yer değiştirme algoritmasıyla karar verilir.

- **LRU (least recently used):** Son zamanlarda en az kullanılan satırın üzerine yazılır.
- **Rastgele yer değiştirme (random replacement).**

Dezavantaj: Verinin hangi satıra yazıldığı bilinemez. Bu yüzden işlemcinin adres yoluna koyduğu adres, önbelleğin bütün satırlarındaki tag alanlarıyla karşılaştırılmalıdır. Karşılaştırma ardışıl yapılırsa çok uzun sürer. Bu yüzden karşılaştırma paralel yapılır ve satır sayısı kadar karşılaştırma devresi gerekir. Örneğin 1024 satır için 1024 devre gerekir. Maliyet bu yüzden artar. Hoca, bu yüzden bu yapıyı "çok sevmedik" dedi.

13.1 Eşleştirme lojiği (match logic)

İki biti karşılaştırmak için en uygun kapı XOR'dur.

Soru: (Hoca sordu.) XOR kapısının iki girişi aynıysa çıkış ne olur?

Cevap: (Bir öğrenci yanıtladı.) Sıfır.

Adres yolundaki bit ile önbellek satırındaki tag biti eşleşiyorsa XOR çıkışı 0 olur. Bütün XOR çıkışları aynı anda 0 iken NOR kapısının çıkışı 1 olur ve eşleşme olduğunu söyler. Dört tag biti için devre dört XOR ve bir NOR kapısından oluşur. Python ile 4 bitlik 256 değer çiftinin hepsinde NOR çıkışının yalnızca iki değer eşitken 1 olduğu doğrulandı.

> **Ders sonrası not:** Hoca devreyi toplarken "dört tane XOR kapısı, bir tane de OR kapısı" dedi. Hemen önceki cümlesindeki gibi son kapı **NOR**'dur.

14. Set Associative Önbellek

Ana bellek aynıdır, yani adres 10 bittir. Önbellek ise *two-way set associative* olarak tasarlanmıştır. Bu yapı, direct mapped yapının çoğullanmış hâlidir. Önbellek toplam 8 satırdır, ama iki parçaya (set 0 ve set 1) bölünmüştür. Her birinde 4 satır vardır.

- **Byte offset:** 2 bit.
- **Index:** Her parça 4 satır olduğu için $\log_2 4 = 2$ bit.
- **Tag:** $10 - 2 - 2 = 6$ bit. Direct mapped'e göre 1 bit artmıştır.

Tag (6 bit)	Index (2 bit)	Byte offset (2 bit)

Şekildeki bir adres, index'ine göre önbelleğin 0. satırına ve o satırın bir bölümüne yazılır. Yüksek anlamlı 6 bit tag olarak kaydedilir. Başka bir adresin index'i 3 ise 3. satıra gider. Hoca verinin hangi parçaya yerleştirildiği konusunda şekildeki set 0 ve set 1 adlarının farklı da sıralanmış olabileceğini söyledi.

Soru: Bu yapının faydası aynı index'li adresleri tutabilmemiz mi?

Cevap: Kesinlikle. Set associative yapının tek avantajı, aynı index'e sahip birden fazla adresi önbellekte tutabilmesidir.

Soru: Kaç set kullanılabiliyor, yalnızca iki mi?

Cevap: Hayır. 4 yollu ve 8 yollu (8-way) yapılar olabilir. Hoca 8'e kadar gördüğünü söyledi.

Soru: Önbelleğin her satırının boyu veri artı adres gibi bir şey mi?

Cevap: Evet, doğru. Ama önbelleğin kapasitesi söylenirken her zaman yalnızca veri alanının büyüklüğü söylenir. Tag için kullanılan alan bu hesaba katılmaz. Verinin önbellekte olup olmadığının anlaşılması için veriyi adresiyle birlikte tutmak gerekir.

14.1 Kaç karşılaştırma devresi gerekir?

Soru: (Hoca sordu.) Fully associative yapıda satır sayısı kadar karşılaştırma devresi gerekiyordu. Two-way set associative yapıda kaç tane gerekir?

Cevap: (Bir öğrenci yanıtladı.) Set sayısı kadar.

Hoca bunu şöyle gerekçelendirdi. Erişilmek istenen her adresin index alanı bilinir ve önbelleğin ilgili satırı bu index ile bulunur. O satırda iki tag alanı vardır. Adres yolundaki tag bu ikisiyle karşılaştırılır. Yani iki eşleştirme devresi yeter.

Kısmi donanım şemasında (*partial hardware block diagram*) önbelleğin içinde tag memory 0, tag memory 1 ve veri alanları vardır. CPU'nun gönderdiği adres tag ve index alanlarına ayrılır. Tag, iki bloğun tag alanıyla karşılaştırılır. İki match logic çıkışı bir OR kapısına girer. OR çıkışı 1 ise hit, 0 ise miss olur. O satırda tag tutmuyorsa aranan adres önbellekte yoktur, çünkü index'e göre başka satıra bakılmasına gerek yoktur.

> **Ders sonrası not:** Hoca paralel bölümlere "set" diyerek "set sayısı kadar karşılaştırıcı" dedi. Altyazıdan doğrulanmayan bir bilgi olarak, büyük olasılıkla yaygın terminolojide aynı index'teki satırların grubuna set, paralel bölümlerin her birine way denir. Bu okumayla örnek 4 set ve 2 way'dir ve karşılaştırıcı sayısı way sayısına eşittir. İki eşleştirme devresi sonucu terimden bağımsızdır.

15. Word ve Blok Kavramları

Ana bellekte bir satırda 4 byte tutulması rastlantı değildir. Bu değer işlemcinin veri yolu genişliği kadardır ve buna **word** denir. 8086'nın adres yolu 20 bit, veri yolu 16 bittir. Bu yüzden 8086'da bir word 16 bit diye öğrenilmişti. Bu örnekteki işlemcinin veri yolu 32 bit olduğu için burada bir word 32 bit, yani 4 byte'tır.

Şimdiye kadar veri önbelleğe hep bir word olarak taşındı. Veri bir **blok (block, line)** olarak da taşınabilir. Blok boyutu (block size) tanımlanır. Bir blok örneğin 2, 4 ya da 8 word'den oluşabilir. Hocaya göre bu, daha verimli bir önbellek kullanımı sağlar.

> **Ek bilgi (derste anlatılmadı):** Blokla taşımanın neden verimli olduğu bu derste açıklanmadı. Büyük olasılıkla neden şudur: Programlar bir adrese eriştikten sonra çoğu zaman yakın adreslere de erişir. Blok bir kez taşınınca bu sonraki erişimler hit olur.

Örnek: Adres yolu 10 bittir. Önbellek two-way set associative'dir, 4 satırlıdır. Bir blok 4 word, bir word 4 byte'tır.

- **Byte offset:** Bir word 4 byte olduğu için $\log_2 4 = 2$ bit.
- **Word offset:** Bir blok 4 word olduğu için $\log_2 4 = 2$ bit.
- **Index:** Önbellek 4 satır olduğu için $\log_2 4 = 2$ bit.
- **Tag:** $10 - (2 + 2 + 2) = 4$ bit.

Tag (4 bit)	Index (2 bit)	Word offset (2 bit)	Byte offset (2 bit)

Şekildeki yapıda set 0 için tag 0 ve data 0, set 1 için tag 1 ve data 1 alanları vardır. Hoca yapıya önce direct mapped dedi, iki set olduğunu görünce two-way set associative olarak düzeltti.

Şekildeki bir adres taşınırken tag alanına 4 bit yazılır. Örnekte adresin index'i 01, word offset'i 00'dır. Veri bir blok olarak taşınır. Blok taşıma örnekleri sonraki derste çoğaltılacak.

Dersin Toparlaması

Bu derste önce paralel ve seri iletişim ile asenkron çerçevenin yapısı görüldü. Ardından veri transferinin üç yöntemi işlendi: işlemciyi bloke eden program kontrollü transfer, bilgisayarda tercih edilen kesmeli transfer ve büyük bloklar için CPU'yu devre dışı bırakan DMA. Kesmeli transferin gerektirdiği öncelik lojiği için seri çalışan daisy chain ile paralel öncelik donanımı karşılaştırıldı. G/Ç bölümü, adres çözümleme, doğrusal seçim ve seri iletim süresi problemleriyle kapatıldı.

İkinci kısımda önbelleğe girildi. SRAM ve DRAM farkının neden küçük ve hızlı bir ara bellek gerektirdiği, hit ve miss kavramları ve üç eşleme tekniği anlatıldı. Direct mapped yapı ucuzdur, ama aynı index'li adresler çakışır. Fully associative yapı çakışmayı kaldırır, ama satır sayısı kadar karşılaştırıcı ister. Set associative yapı ise aynı index'li birden fazla adresi az sayıda karşılaştırıcıyla tutar. Son olarak word ve blok kavramları eklendi. Sonraki derste önbellek konusu sürecek ve blok taşımayla ilgili örnekler çoğaltılacak.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.